

TECHNICAL ARCHITECTURE

H2O Online System Architecture & User Journey

End-to-end architecture documentation covering AWS deployment, MongoDB Atlas, backend API, authentication, Groq AI, Google Maps, Razorpay, admin portal, address & serviceability flows, order lifecycle across customer · supplier · rider, and platform onboarding journeys.

17

Sections

20

Flow Diagrams

5

User Roles

6

Third-Party APIs

Contents

- | | |
|---------------------------------------|--------------------------------------|
| 1. System Overview | 10. Supplier Onboarding Journey |
| 2. End-to-End Architecture | 11. Rider / Delivery Partner Journey |
| 3. AWS Deployment Architecture | 12. Store Management Journey |
| 4. Third-Party Integrations | 13. Admin Portal Roles |
| 5. Authentication & Authorization | 14. Sample Customer User Journey |
| 6. Address & Serviceability Data Flow | 15. Multi-Actor Order Fulfillment |
| 7. Order Lifecycle — Cross-Actor Flow | 16. Key API Reference by Actor |
| 8. Payment Data Flow (Razorpay) | 17. Data Models (Core Entities) |
| 9. AI (Groq) Data Flow | |

Version: 1.0

Production API: <http://13.62.57.255:5000>

Admin Portal: <http://13.62.57.255:3000>

Database: MongoDB Atlas (H2Online)

Generated: 29 June 2026

H2O Online — System Architecture & User Journey Documentation

Version: 1.0

Last updated: June 2026

Production API: `http://13.62.57.255:5000`

Production Admin: `http://13.62.57.255:3000`

Database: MongoDB Atlas (`H20online`)

This document describes the end-to-end architecture, third-party integrations, data flows (address, order, auth), and user journeys across all actors: **Customer, Supplier, Delivery Partner (Rider), Society, and Admin.**

Table of Contents

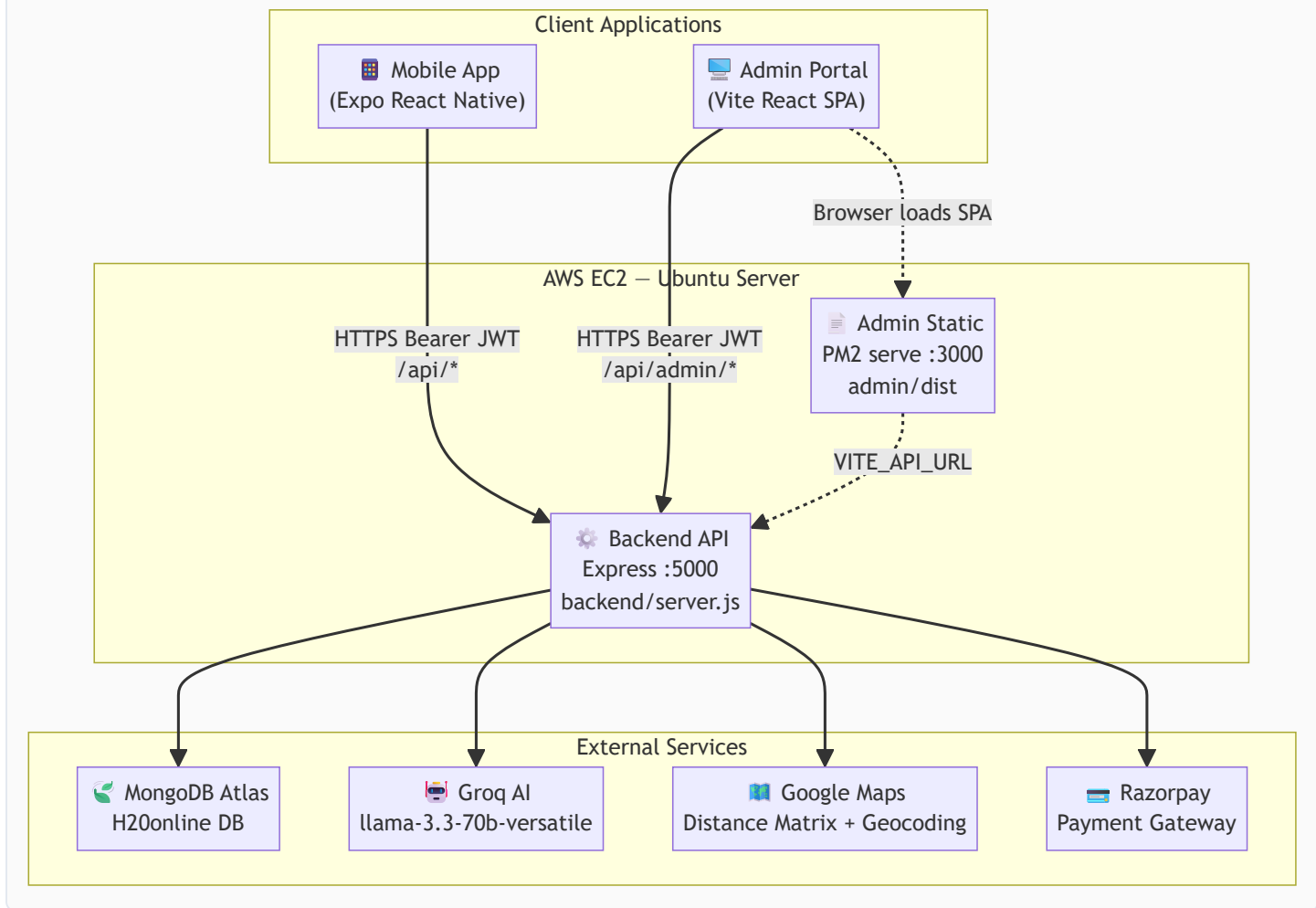
- 1. System Overview
- 2. End-to-End Architecture Diagram
- 3. Deployment Architecture (AWS)
- 4. Third-Party Integrations
- 5. Authentication & Authorization
- 6. Address & Serviceability Data Flow
- 7. Order Lifecycle — Cross-Actor Flow
- 8. Payment Data Flow (Razorpay)
- 9. AI (Groq) Data Flow
- 10. Supplier Onboarding Journey
- 11. Rider / Delivery Partner Journey
- 12. Store Management Journey
- 13. Admin Portal Roles & Capabilities
- 14. Sample Customer User Journey
- 15. Multi-Actor Order Fulfillment Journey
- 16. Key API Reference by Actor
- 17. Data Models (Core Entities)

1. System Overview

H2O Online is a water delivery platform with three client applications and one shared backend:

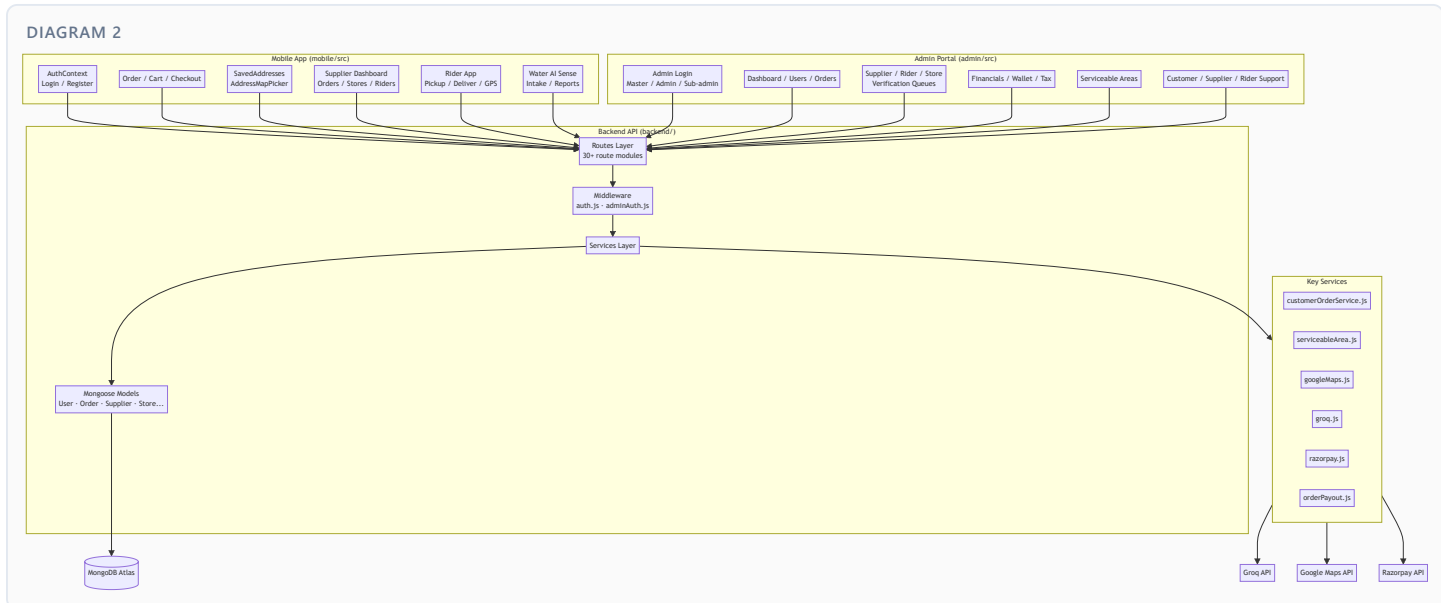
Layer	Technology	Folder	Purpose
Mobile App	Expo / React Native	<code>mobile/</code>	Customer ordering, supplier ops, rider delivery, society accounts
Admin Portal	Vite / React SPA	<code>admin/</code>	Platform management, approvals, financials, support
Backend API	Node.js / Express	<code>backend/</code>	REST API, business logic, third-party orchestration
Database	MongoDB Atlas	Cloud	Persistent storage for all entities
Infrastructure	AWS EC2 (Ubuntu)	<code>config/aws-production.json</code>	Hosts API (:5000) and Admin SPA (:3000) via PM2

DIAGRAM 1

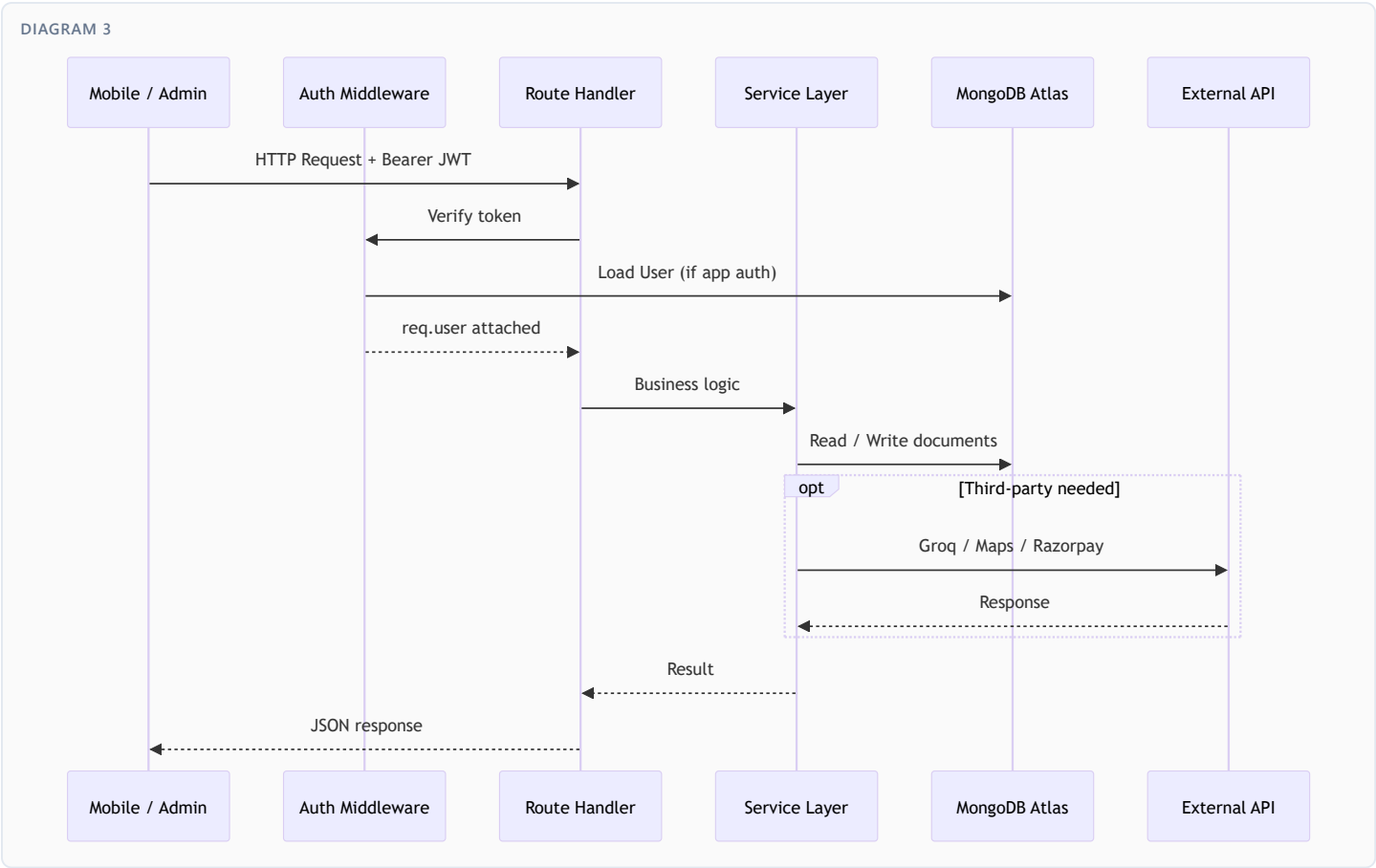


2. End-to-End Architecture Diagram

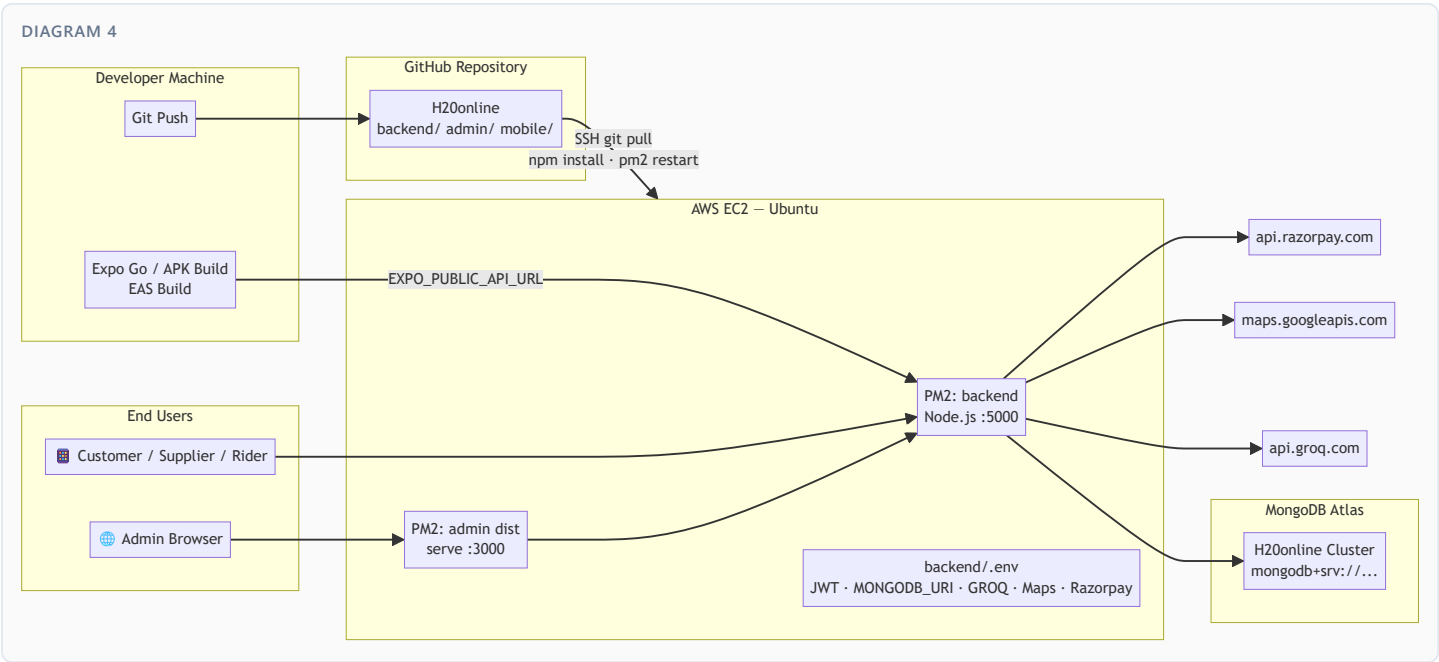
2.1 Logical Component Architecture



2.2 Request Flow (Generic)



3. Deployment Architecture (AWS)



Production Configuration

Component	Config File	Key Variables
Backend	backend/.env	MONGODB_URI , JWT_SECRET , GROQ_API_KEY , GOOGLE_MAPS_API_KEY , RAZORPAY_KEY_ID/SECRET , CORS_ORIGINS
Admin	admin/.env	VITE_API_URL=http://13.62.57.255:5000
Mobile	mobile/.env / eas.json	EXPO_PUBLIC_API_URL , EXPO_PUBLIC_GOOGLE_MAPS_API_KEY , EXPO_PUBLIC_RAZORPAY_KEY_ID
AWS URLs	config/aws-production.json	Single source of truth for EC2 IP and ports

Security Group ports: 22 (SSH), 5000 (API), 3000 (Admin), 80/443 (optional reverse proxy).

4. Third-Party Integrations

Integration	Provider	Config (Backend)	Config (Mobile)	Used For	Backend Files
Database	MongoDB Atlas	MONGODB_URI	—	All persistence	config/db.js , config/mongo.js
AI	Groq	GROQ_API_KEY , GROQ_MODEL	—	Water insights, intake tips, reports, Q&A	services/groq.js , routes/ai.js , services/waterAiContext.js
Maps (server)	Google Maps	GOOGLE_MAPS_API_KEY	—	Distance Matrix (ETA), Geocoding (PIN→coords)	services/googleMaps.js , services/serviceableArea.js , routes/maps.js
Maps (client)	Google Maps	—	EXPO_PUBLIC_GOOGLE_MAPS_API_KEY	Map picker, reverse geocode, device location	mobile/src/components/AddressMapPicker.native.js , utils/googleMaps.js
Payments	Razorpay	RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET	EXPO_PUBLIC_RAZORPAY_KEY_ID	Online payment → order creation	services/razorpay.js , routes/payments.js
Hosting	AWS EC2	Server .env	EXPO_PUBLIC_API_URL	Production API + Admin hosting	docs/DEPLOYMENT-GUIDE.md , PM2

Groq AI Endpoints

Endpoint	Method	Purpose
/api/ai/water-insight	GET	Personalized hydration insight on dashboard
/api/ai/intake-sense?date=	GET	Daily intake analysis
/api/ai/water-report	POST	Generate health/hydration report
/api/ai/ask	POST	Natural language Q&A about water/health

Fallback responses are returned when `GROQ_API_KEY` is missing or the API fails (`services/groq.js`).

Google Maps Endpoints

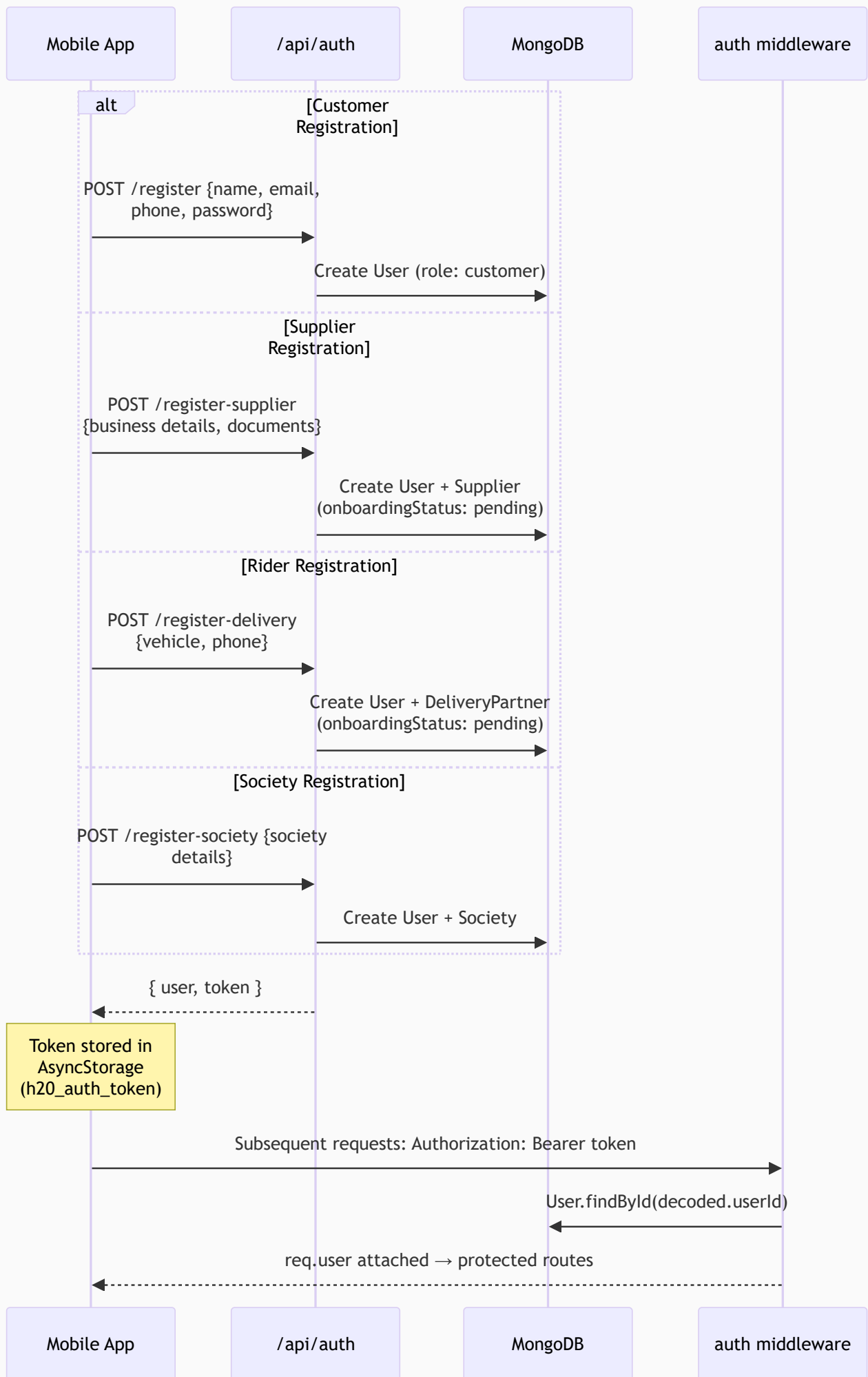
Endpoint	Method	Purpose
/api/maps/travel	POST	Batch distance/duration from store → customer
/api/serviceability/check	POST	PIN/radius serviceability (uses geocoding internally)

5. Authentication & Authorization

5.1 Mobile App Authentication (JWT)

All mobile users authenticate via `POST /api/auth/*` and receive a **7-day JWT** (`{ userId }`).

DIAGRAM 5



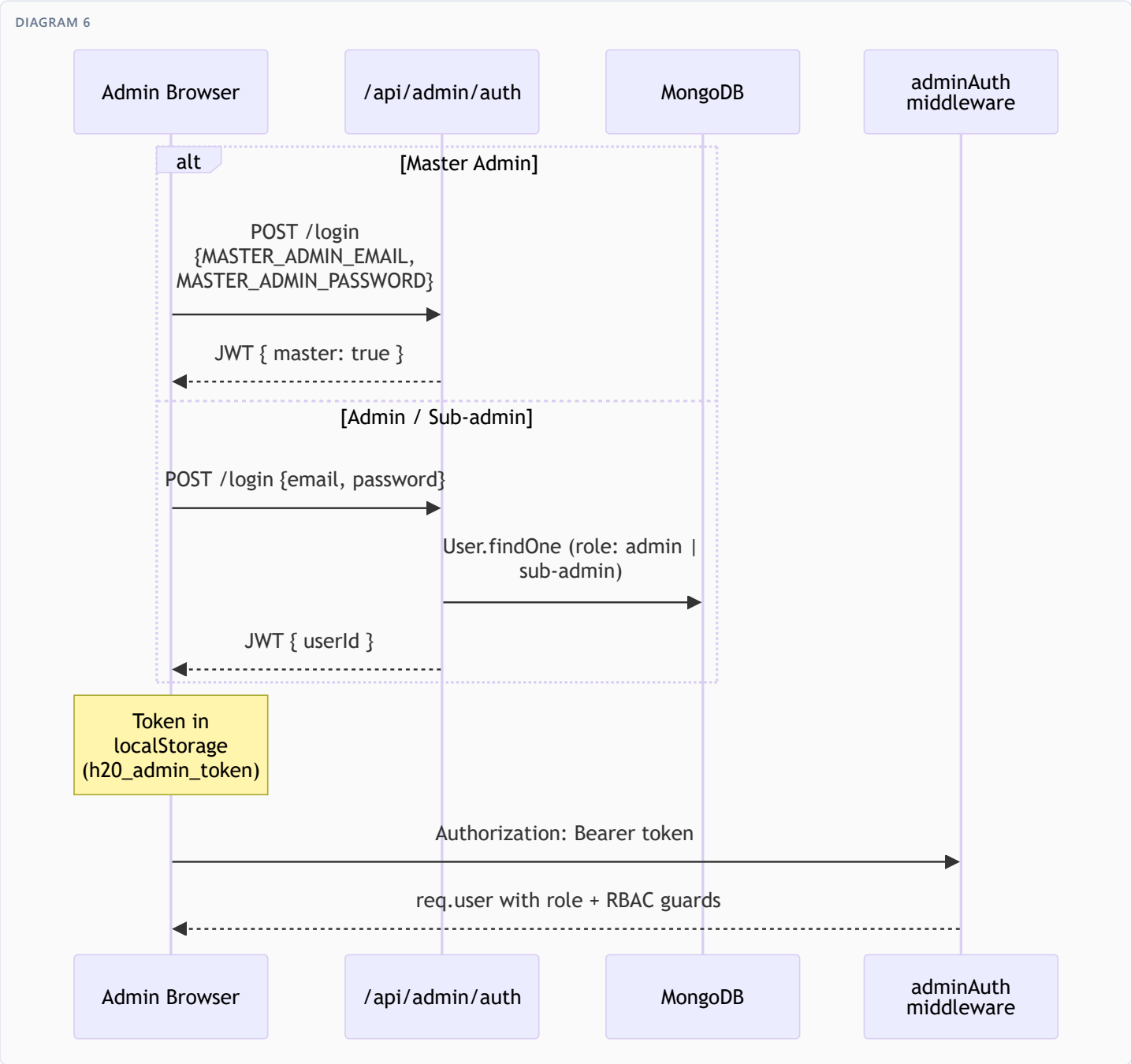
5.2 Mobile User Roles

Role	Registration	Default Status	Mobile Screens After Login
customer	POST /api/auth/register	Active	Dashboard, Order, Checkout, Tracking
supplier	POST /api/auth/register-supplier	Pending until admin verify	Verification pending → Supplier dashboard
deliveryPartner	POST /api/auth/register-delivery	Pending until admin verify	Verification pending → Rider dashboard
society	POST /api/auth/register-society	Active	Society product catalog & orders

Session restore: App reads token from AsyncStorage → GET /api/users/me on startup (mobile/src/context/AuthContext.js).

Routing logic: mobile/src/utils/authRouting.js redirects suppliers/riders with onboardingStatus !== "approved" to verification-pending screens.

5.3 Admin Portal Authentication



5.4 Admin Role Permissions (RBAC)

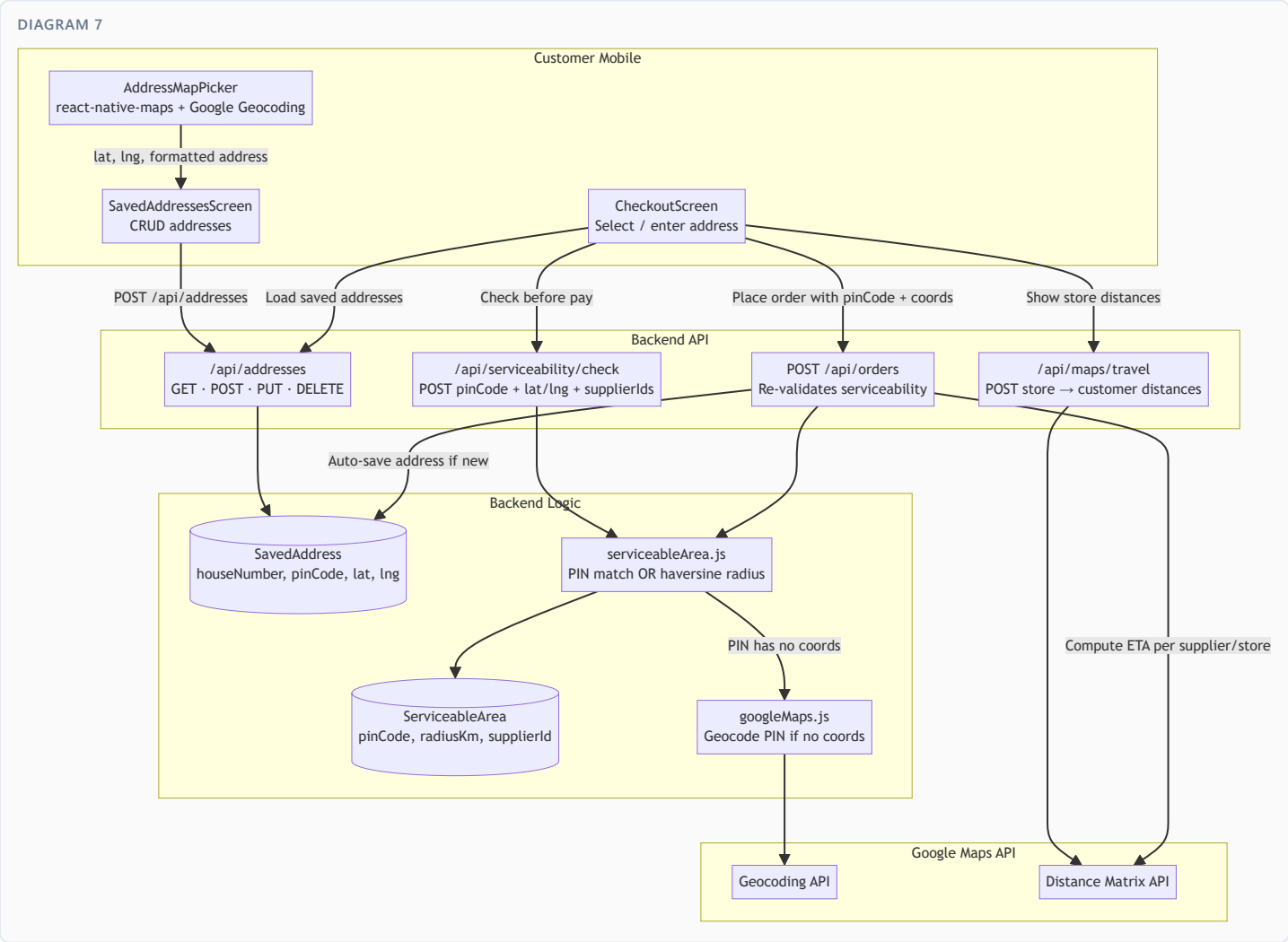
Capability	Master	Admin	Sub-admin
Dashboard, Users, Orders (view)	✓	✓	✓
Supplier / Rider / Store verify	✓	✓	✓
Products, Plans, Subscriptions	✓	✓	✓
Serviceable Areas	✓	✓	✓
Support tickets (all types)	✓	✓	✓

Capability	Master	Admin	Sub-admin
Surveys	✓	✓	✓
Financials	✓	✓	✗
Delete user	✓	✓	✗
Remove supplier	✓	✓	✗
Create admin users	✓	✓	✗
Change master credentials	✓	✗	✗

Guards implemented in `backend/middleware/adminAuth.js`: `requireCanSeeFinancials`, `requireCanDeleteUser`, `requireCanRemoveSupplier`, `requireCanCreateAdmin`.

6. Address & Serviceability Data Flow

Addresses and serviceability are checked at **three layers**: map picker (client), checkout (client + API), and order creation (server — authoritative).



Serviceability Rules

- Admin defines **ServiceableArea** per supplier: PIN code + optional lat/lng + radius (default 10 km).
- `checkServiceability()` matches if:
 - Exact **6-digit PIN** matches, **OR**
 - Customer coordinates fall within **haversine radius** of supplier area center.
- If PIN has no coordinates, backend **geocodes** via Google Maps.
- Checkout calls `POST /api/serviceability/check` — blocks order if unserviceable.
- `createCustomerOrder()` **re-checks server-side** (client check is not trusted alone).

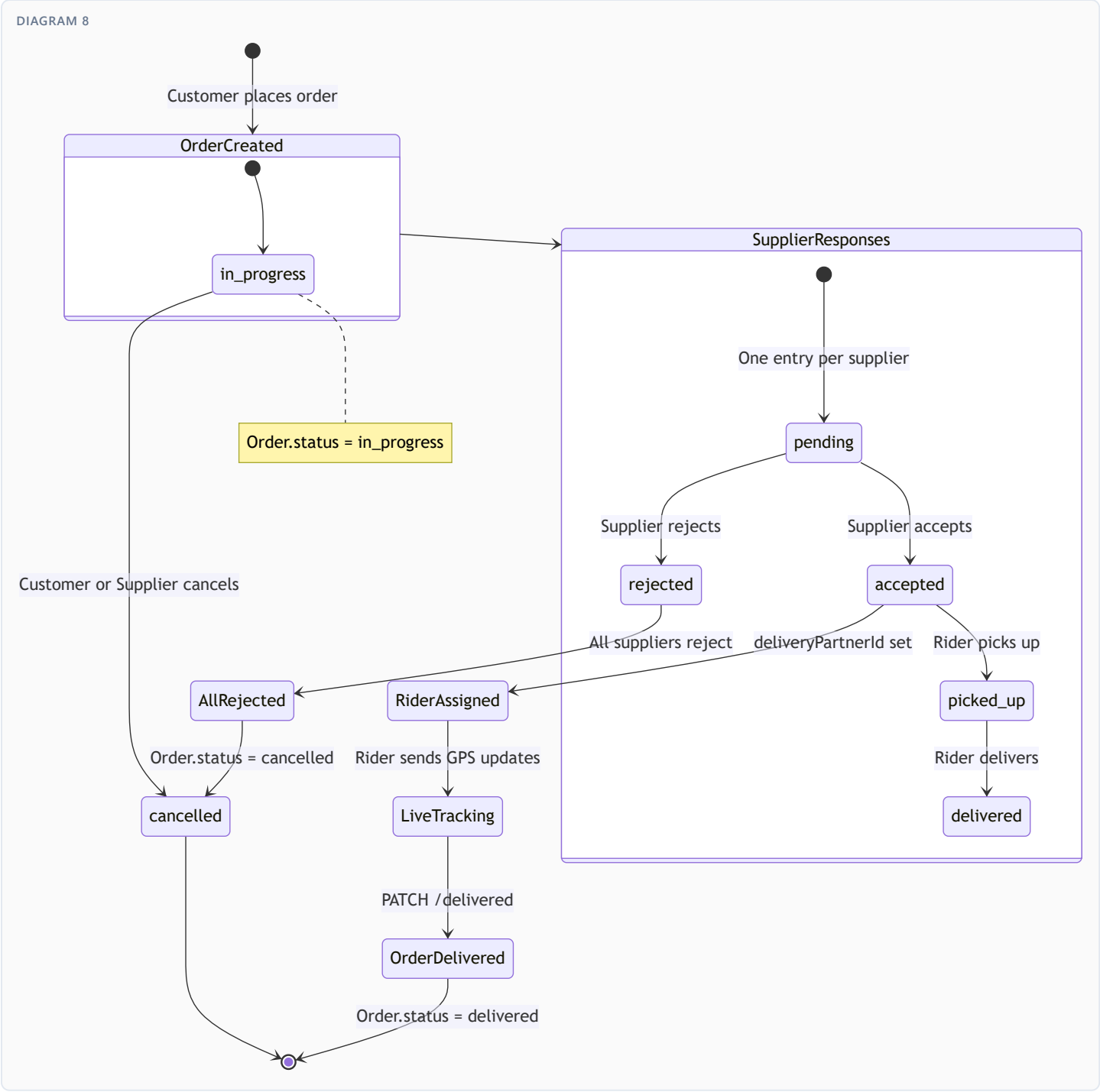
Address Fields (SavedAddress model)

Field	Required	Used In
houseNumber, locality, city, state	Yes	Display, order delivery

Field	Required	Used In
pinCode (6-digit)	Yes	Serviceability
phoneNumber	Yes	Delivery contact
latitude, longitude	Optional (from map picker)	Radius serviceability, travel ETA
isDefault	Optional	Checkout pre-selection

7. Order Lifecycle — Cross-Actor Flow

7.1 Order State Machine



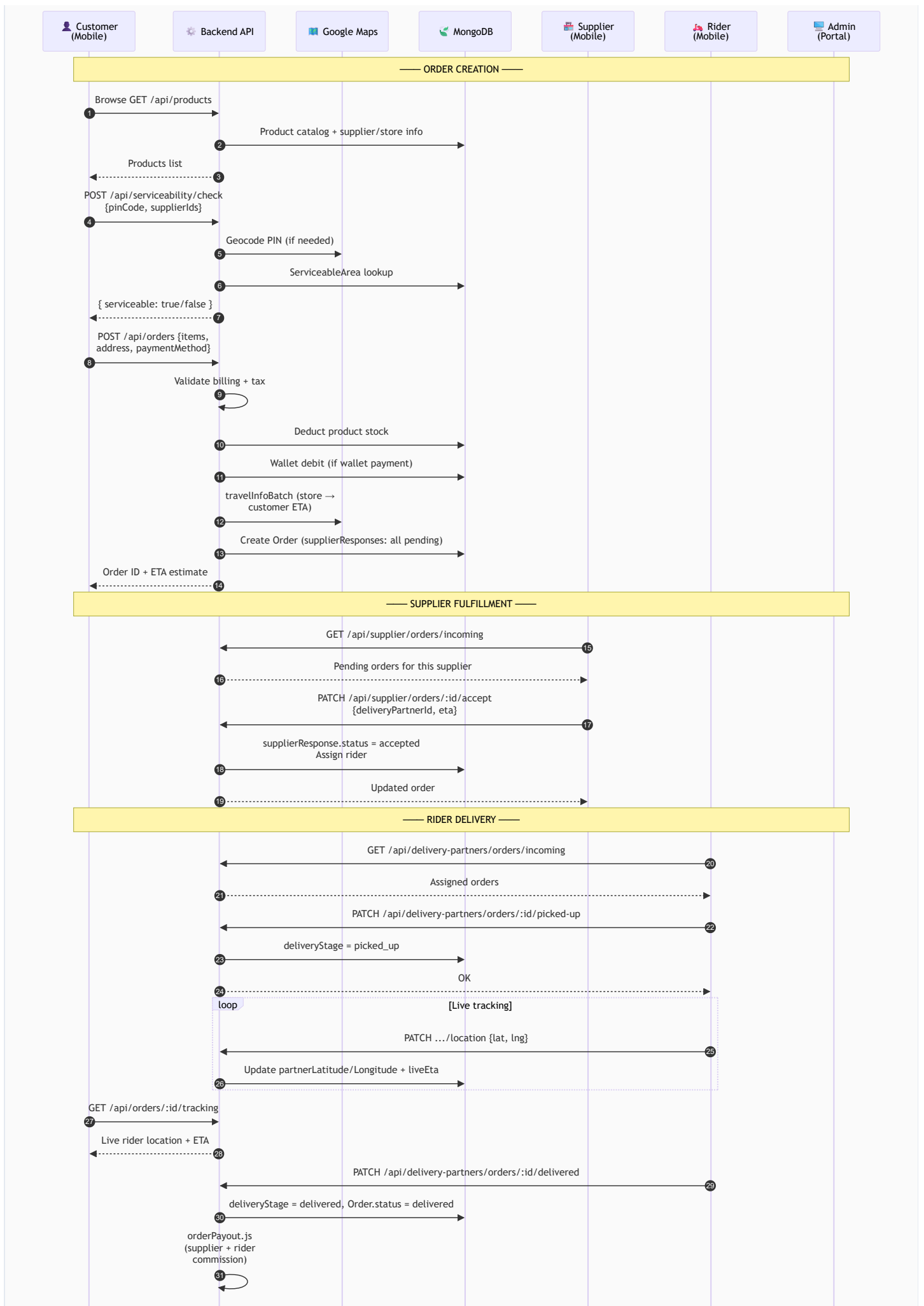
7.2 Order Status Fields

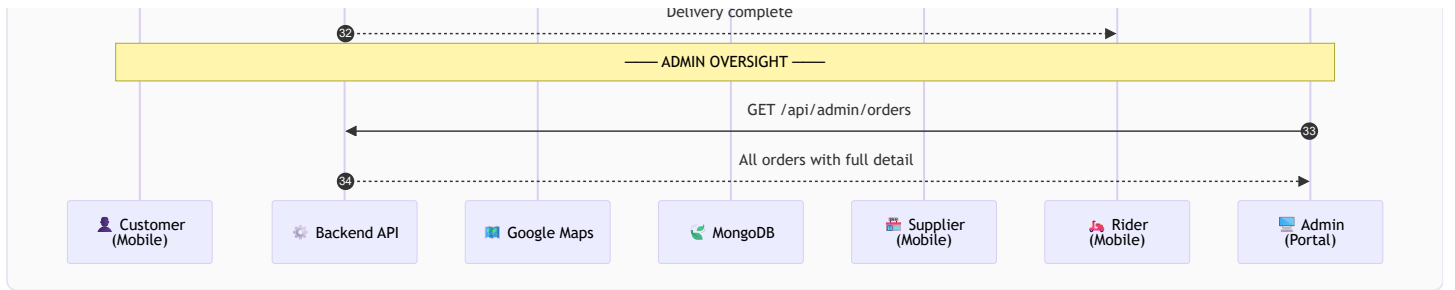
Field	Values	Actor	
Order.status	in_progress → delivered \	cancelled	System
supplierResponses[].status	pending → accepted \	rejected	Supplier
supplierResponses[].deliveryStage	accepted → picked_up → delivered	Rider	

Field	Values	Actor		
paymentStatus	pending \	paid \	failed	Payment gateway

7.3 End-to-End Order Sequence (All Actors)

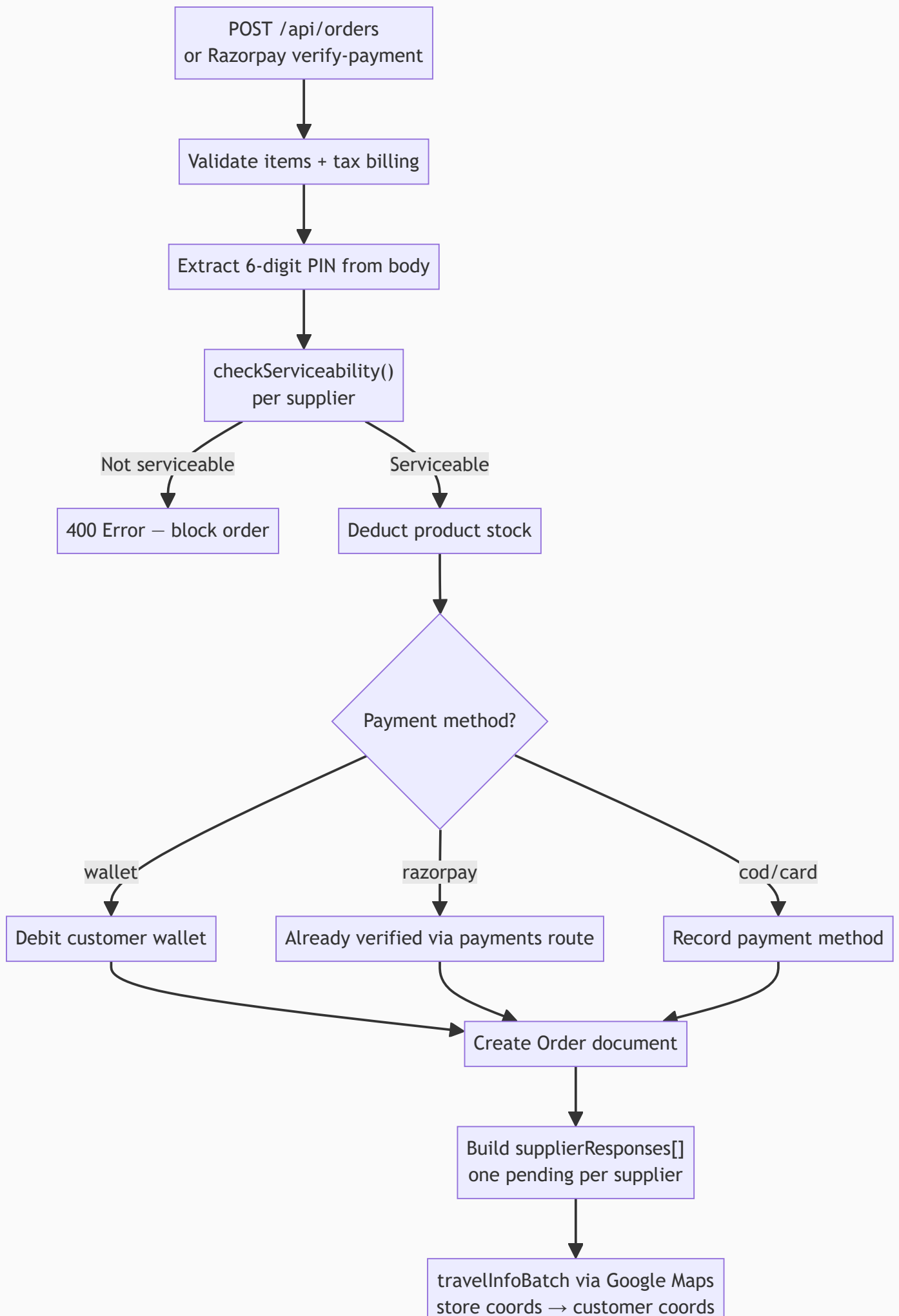


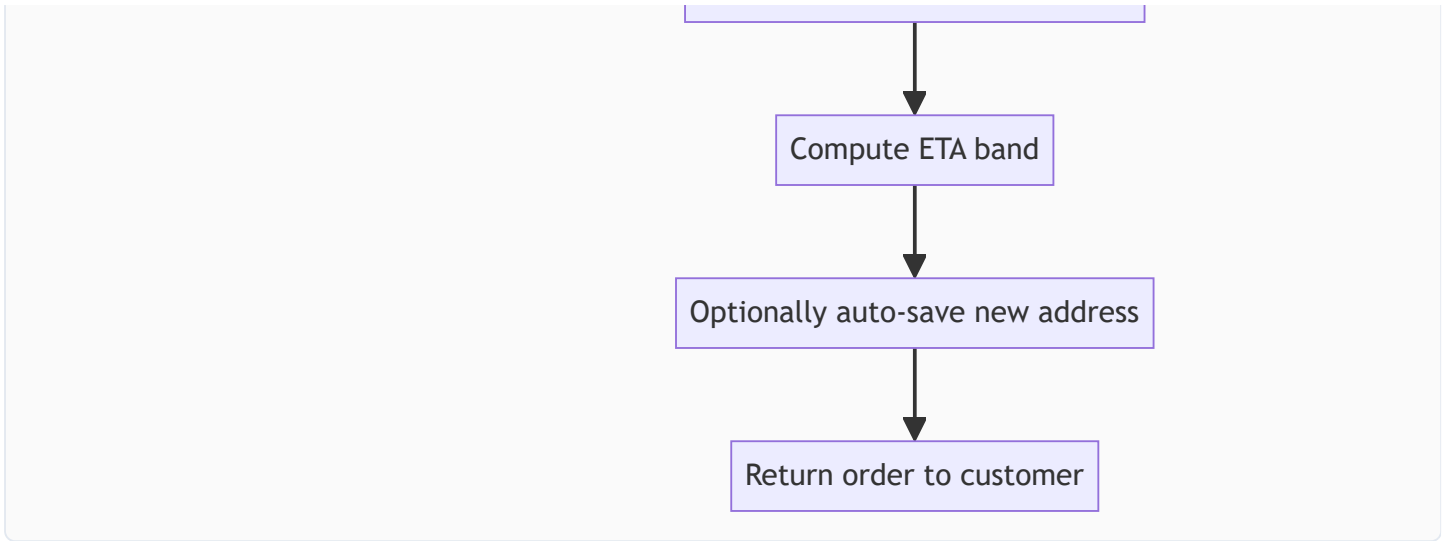




7.4 Order Creation Internals (customerOrderService.js)

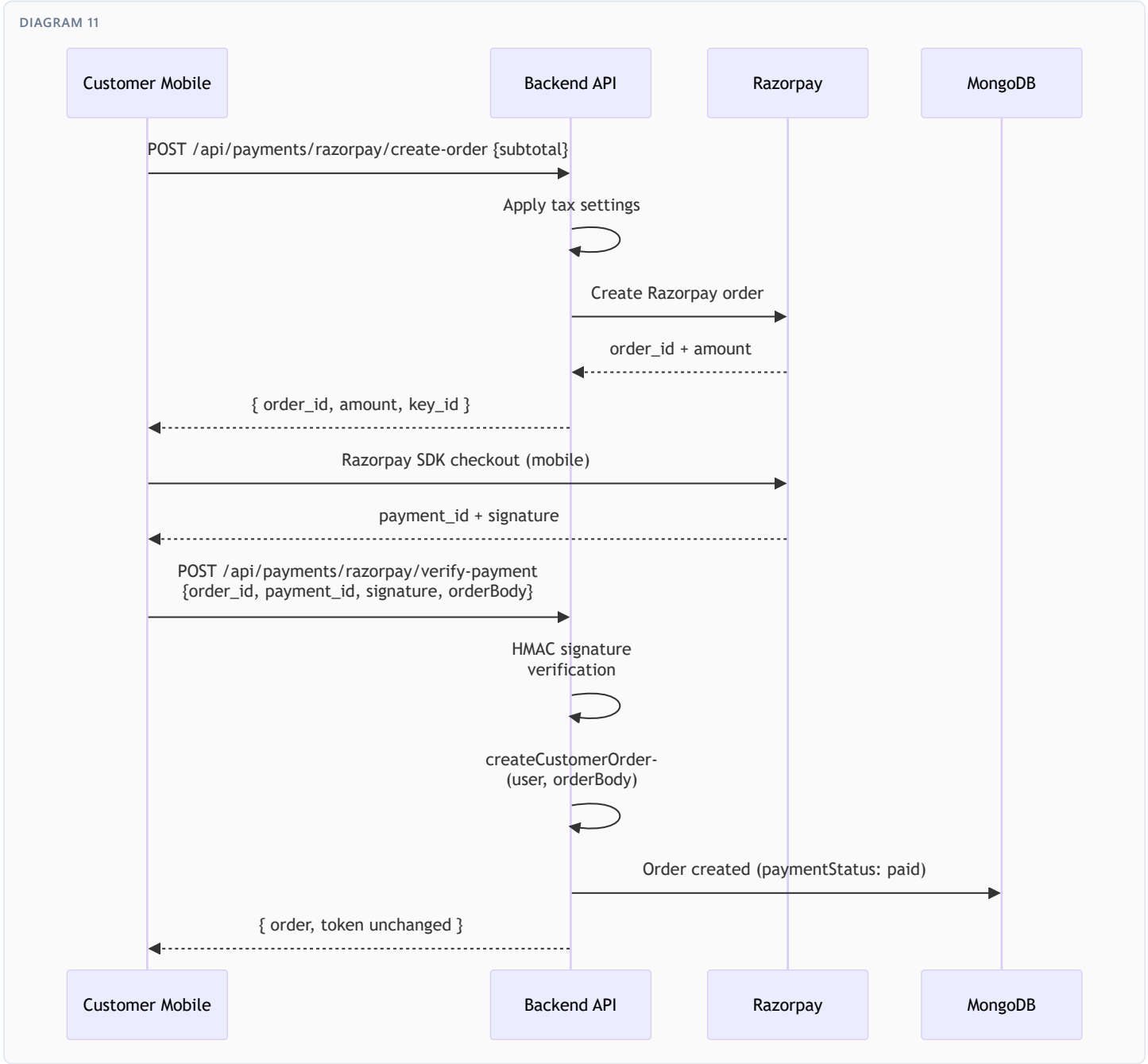






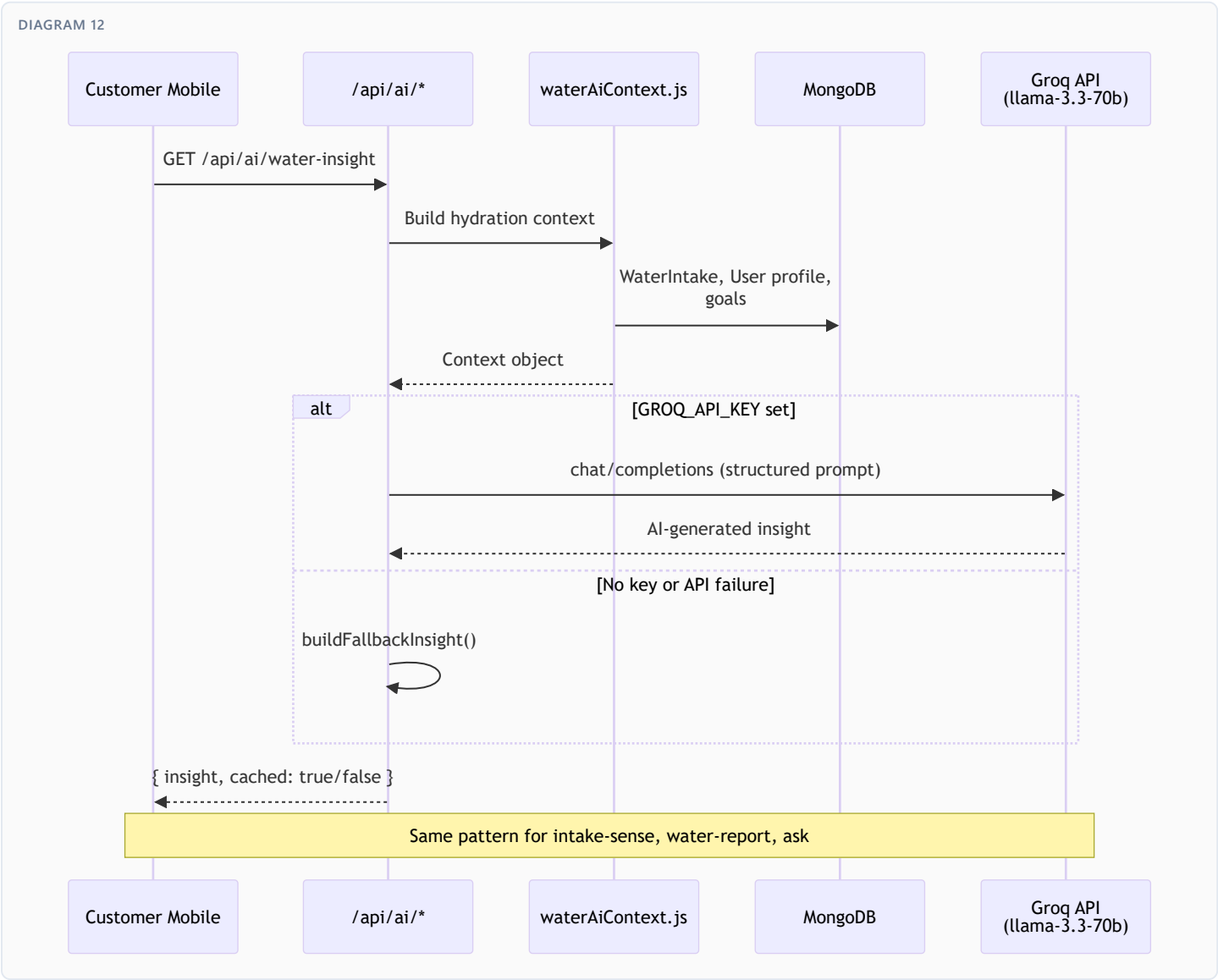
8. Payment Data Flow (Razorpay)

Direct `POST /api/orders` with `paymentMethod: "razorpay"` is **rejected** — payment must go through the verify flow.



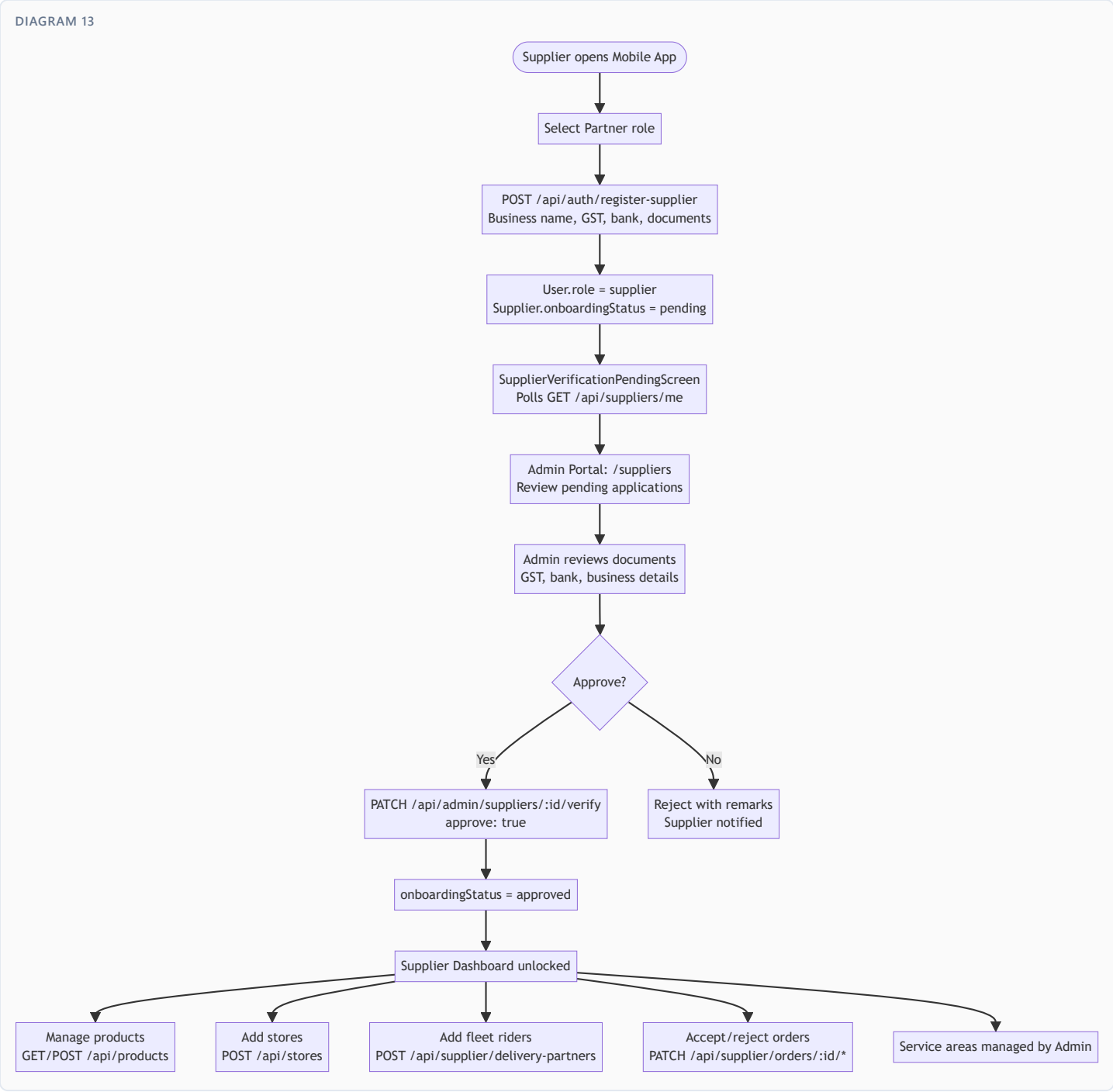
Other payment methods: `wallet` (debit on create), `cod`, `card` (direct order create).

9. AI (Groq) Data Flow



Caching: 15-minute in-memory cache per user per insight type (`services/groq.js`).

10. Supplier Onboarding Journey



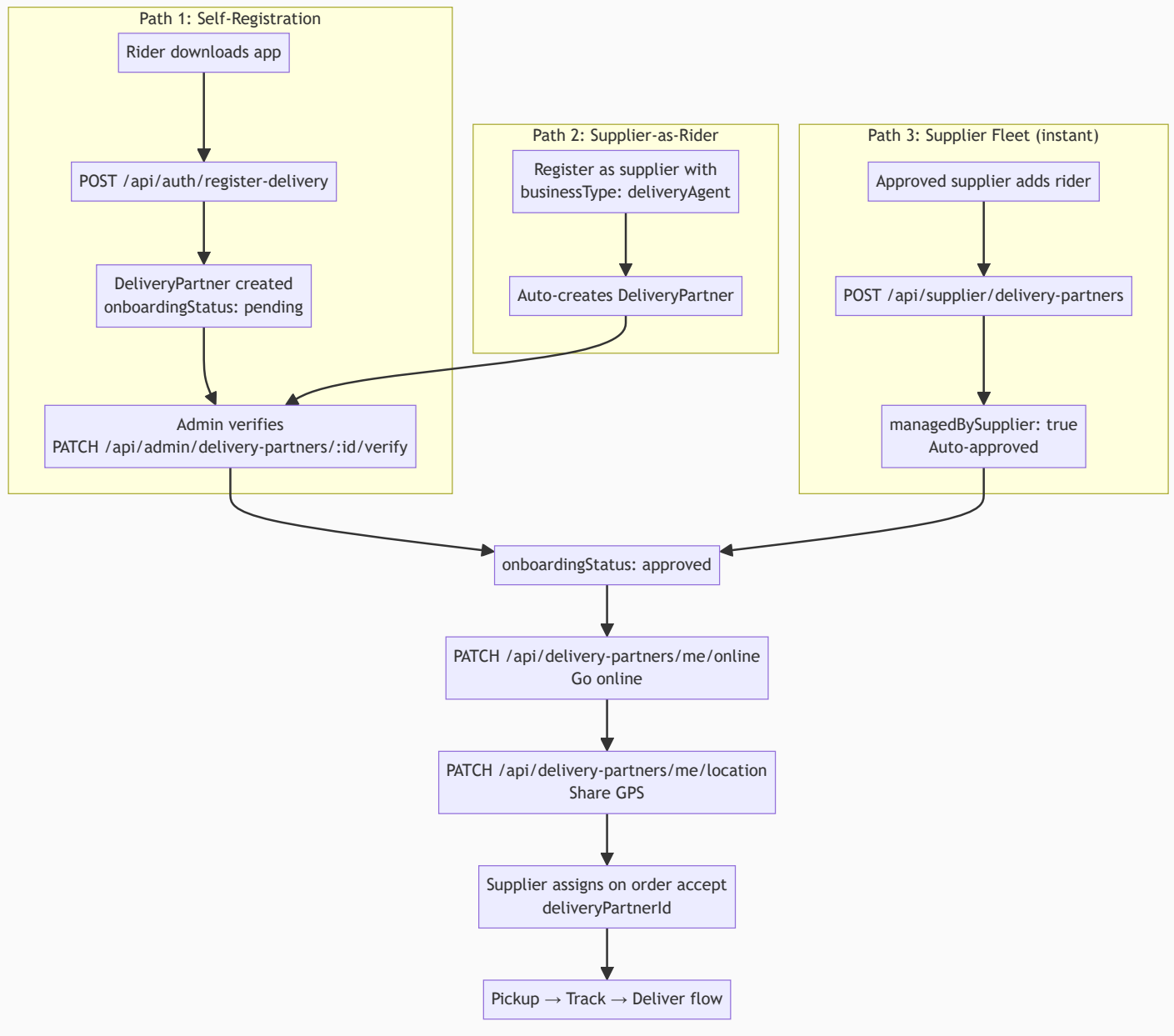
Supplier Registration Data

Field	Stored In	Verified By
Business name, contact	Supplier	Admin
GST number	Supplier	Admin
Bank details	Supplier	Admin
Document URLs (license, etc.)	Supplier.documents	Admin
Business address + lat/Ing	Supplier	Admin / self-update via PATCH /api/suppliers/me

11. Rider / Delivery Partner Journey

Riders can join the platform through **three paths**:

DIAGRAM 14



Rider Vehicle Types

bike · van · bicycle · tanker · miniTruck

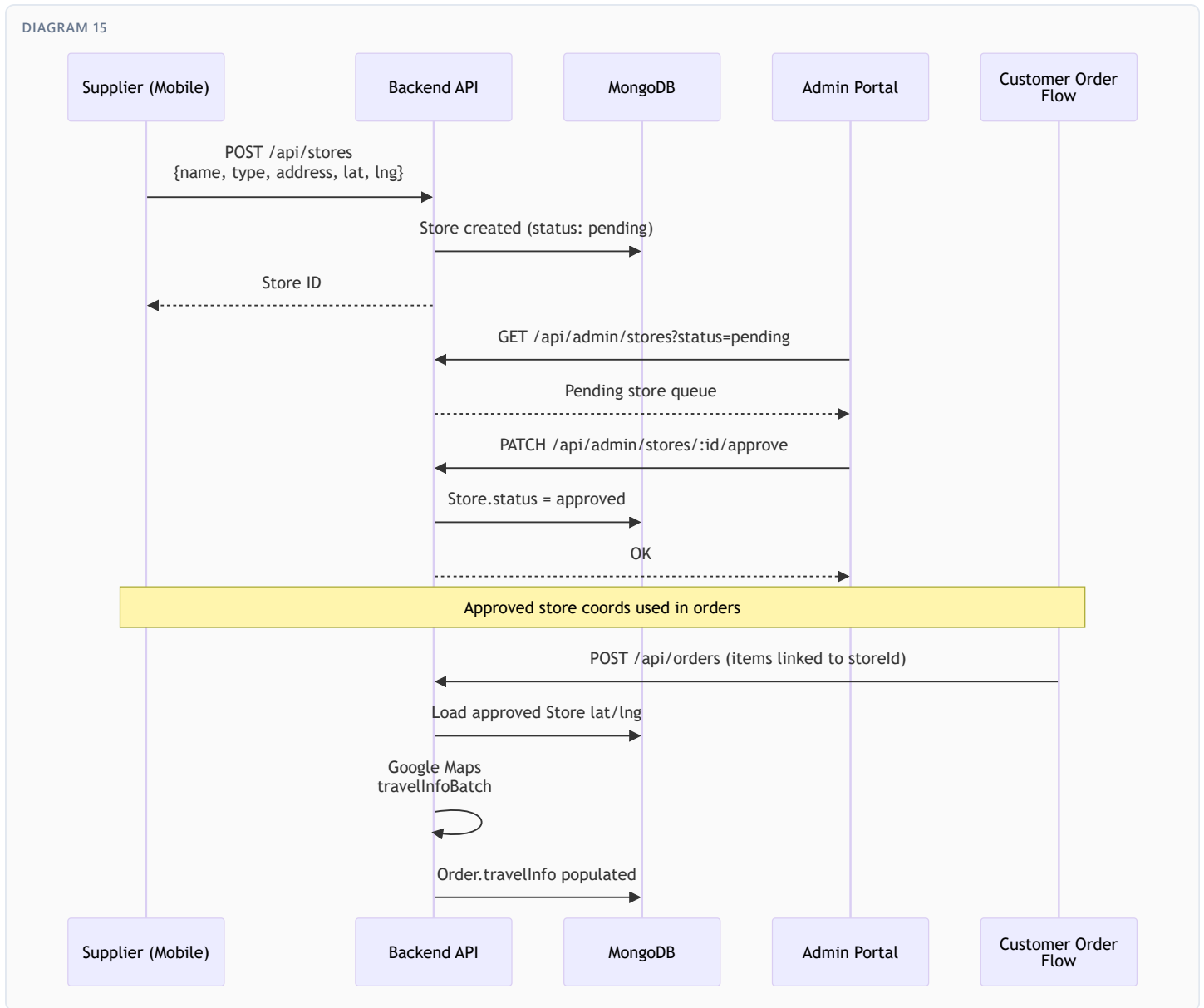
Rider Availability Rules (`partnerAvailability.js`)

- Must be **admin-approved** (`onboardingStatus: approved`)
- Must be **online** to receive assignments
- Cannot be on an **in-flight delivery** for another order

12. Store Management Journey

Stores/warehouses are the physical fulfillment points used for travel ETA and order routing.

DIAGRAM 15



Store Field	Purpose	
type	store or warehouse	
latitude, longitude	Required — map pin on creation	
status	pending → approved \	rejected
supplierId	Owner supplier	

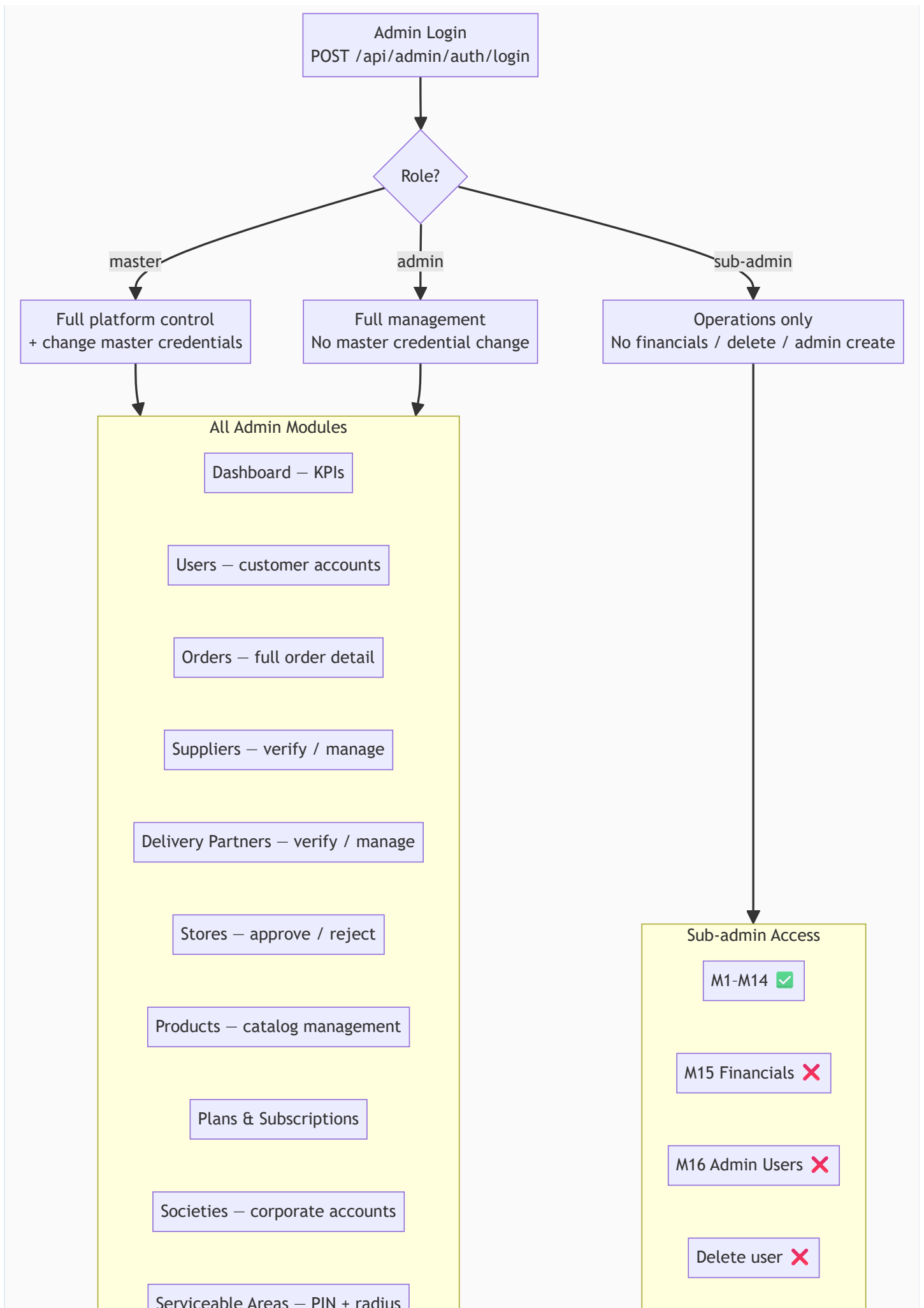
Edit rule: If an approved store is edited by supplier, it may return to **pending** for re-approval.

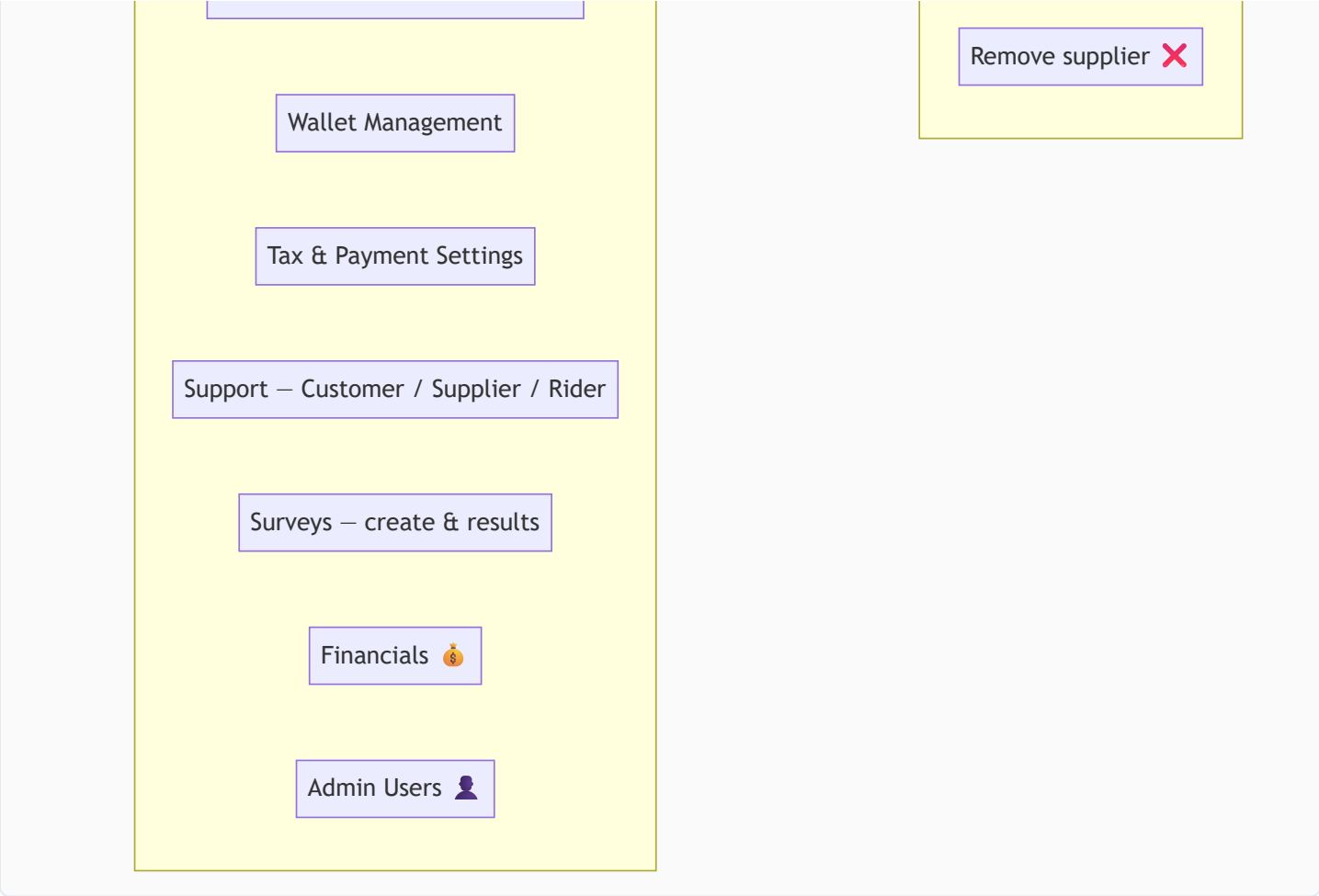
Mobile screens: `SupplierStoresScreen.js`, `SupplierStoreLocationScreen.js`

Admin screen: `admin/src/pages/Stores.jsx`

13. Admin Portal Roles & Capabilities







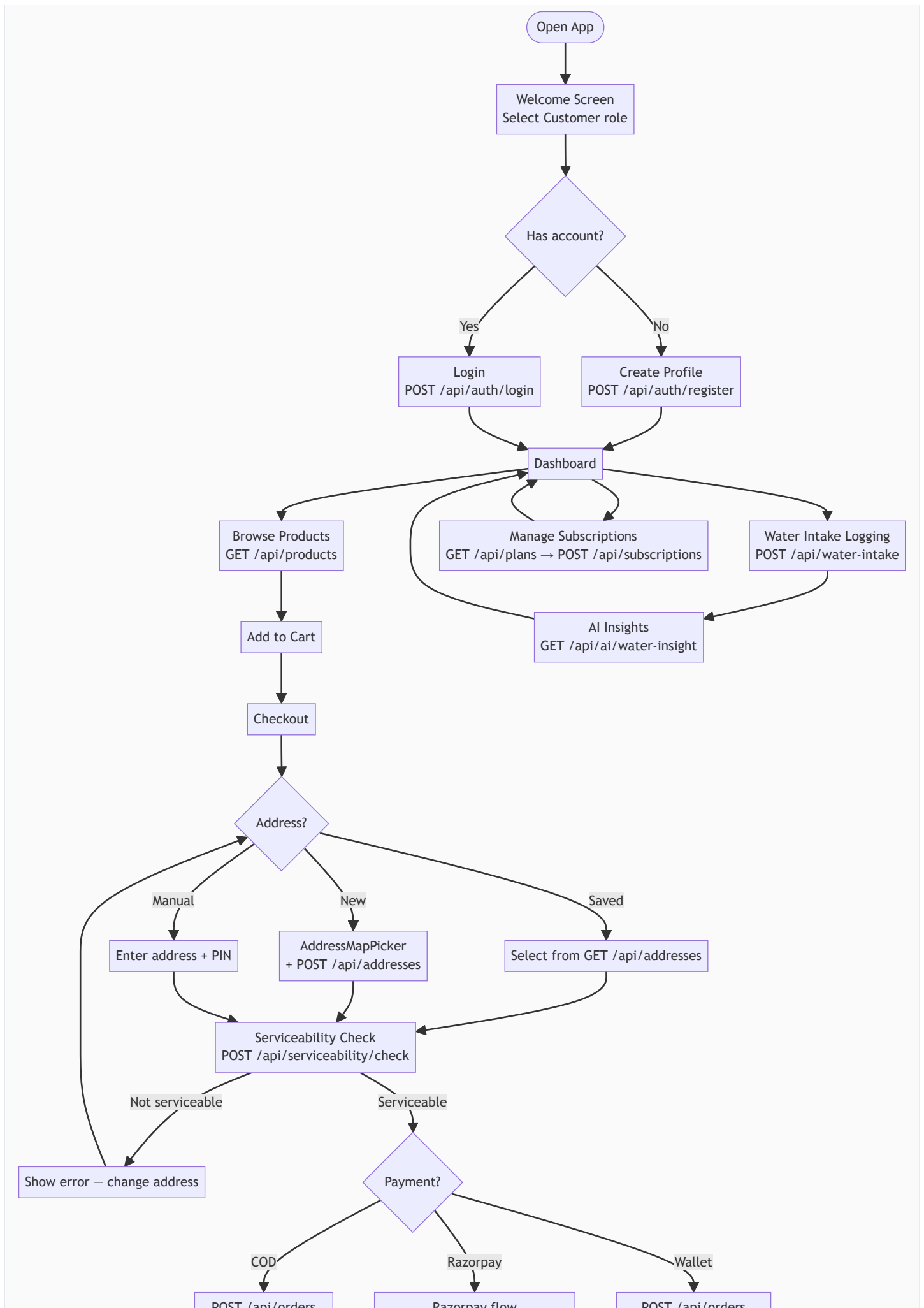
Admin ↔ Platform Operations Map

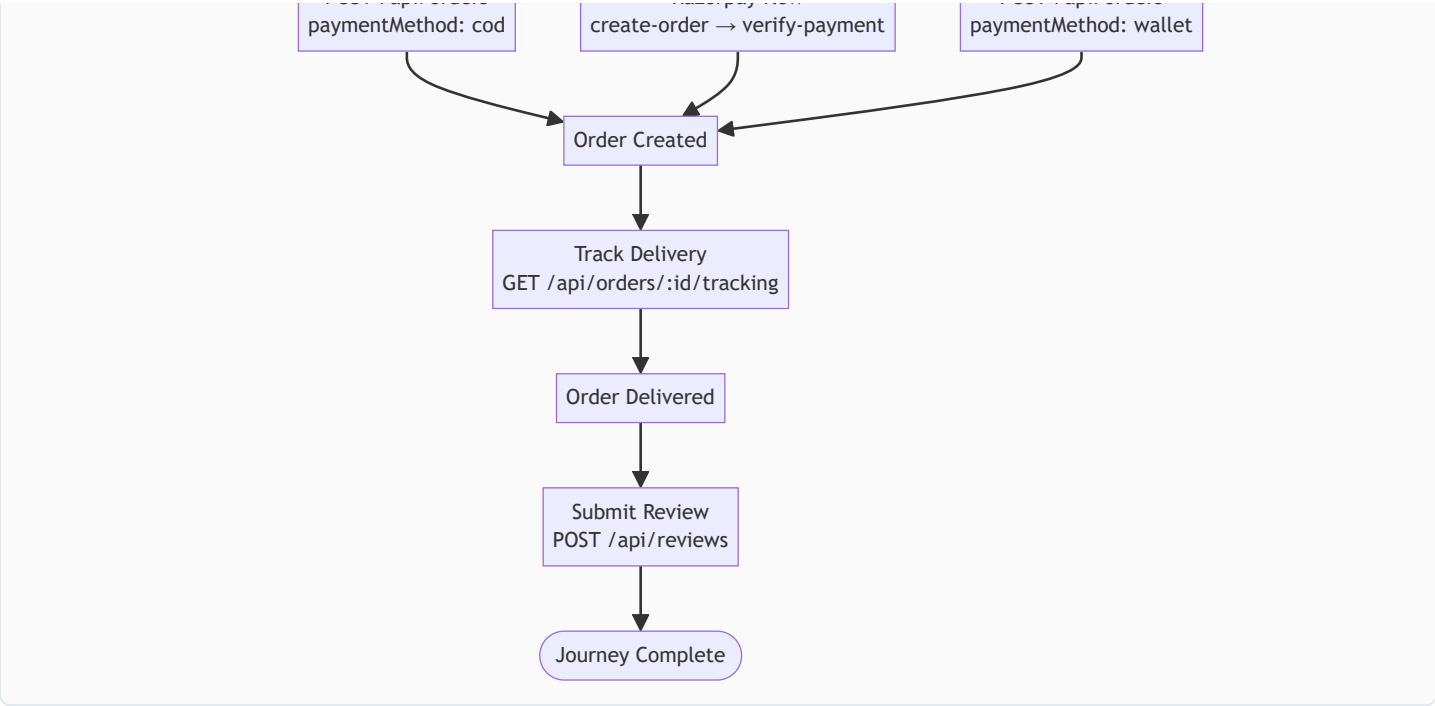
Admin Action	Affects	Mobile Impact
Verify supplier	Supplier onboarding	Supplier dashboard unlocks
Verify rider	Rider onboarding	Rider can go online
Approve store	Store status	Store coords used in order ETA
Manage serviceable areas	ServiceableArea records	Checkout serviceability check
Tax settings	PlatformSettings	Order totals, Razorpay amounts
Wallet management	Wallet balances	Customer wallet checkout
Product management	Product catalog	Customer product browse

14. Sample Customer User Journey

14.1 Customer Journey Flow Diagram





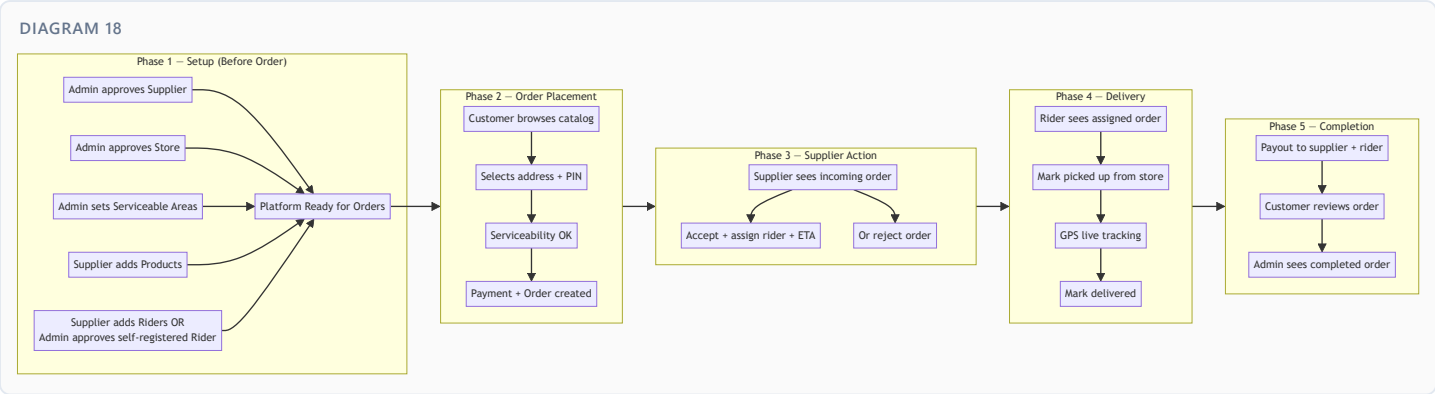


14.2 Customer Journey Steps (Detailed)

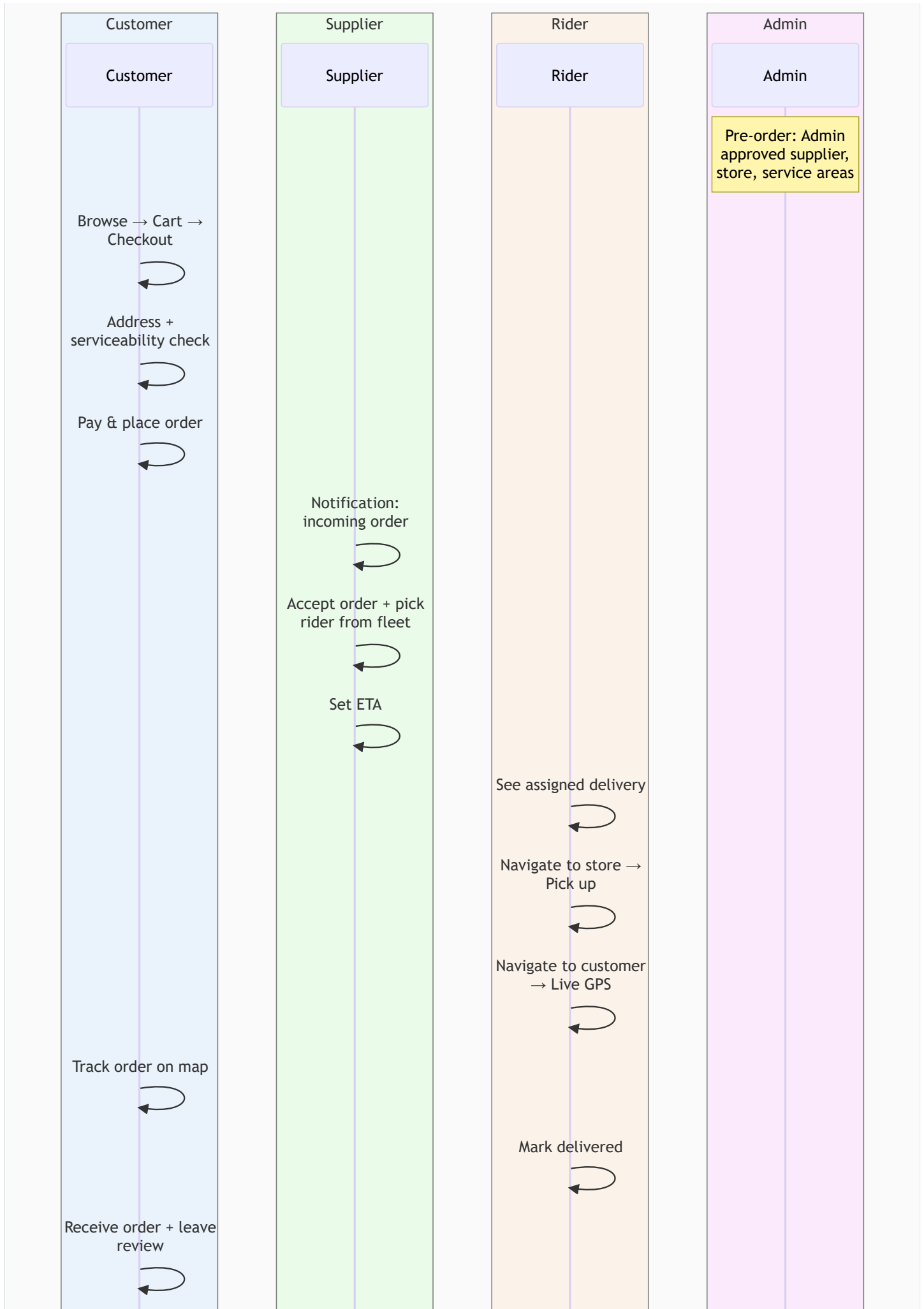
Step	Screen	API / Integration	Outcome
1	Welcome & Role Selection	—	Choose Customer path
2	Create Profile / Login	POST /api/auth/register or /login	JWT token stored
3	Dashboard	GET /api/users/me , /api/wallet , /api/ai/water-insight	Personalized home + Groq insight
4	Browse Products	GET /api/products	Catalog with supplier info
5	Cart	Local state	Review items
6	Checkout — Address	GET /api/addresses , Map picker + Google Maps	Delivery location set
7	Checkout — Serviceability	POST /api/serviceability/check + Google Geocoding	Confirm deliverable
8	Checkout — Payment	Wallet / Razorpay / COD	Payment authorized
9	Order Placed	POST /api/orders or Razorpay verify	Order in_progress
10	Track Delivery	GET /api/orders/:id/tracking	Live rider GPS + ETA
11	Delivered	—	Status delivered
12	Review	POST /api/reviews	Product rating

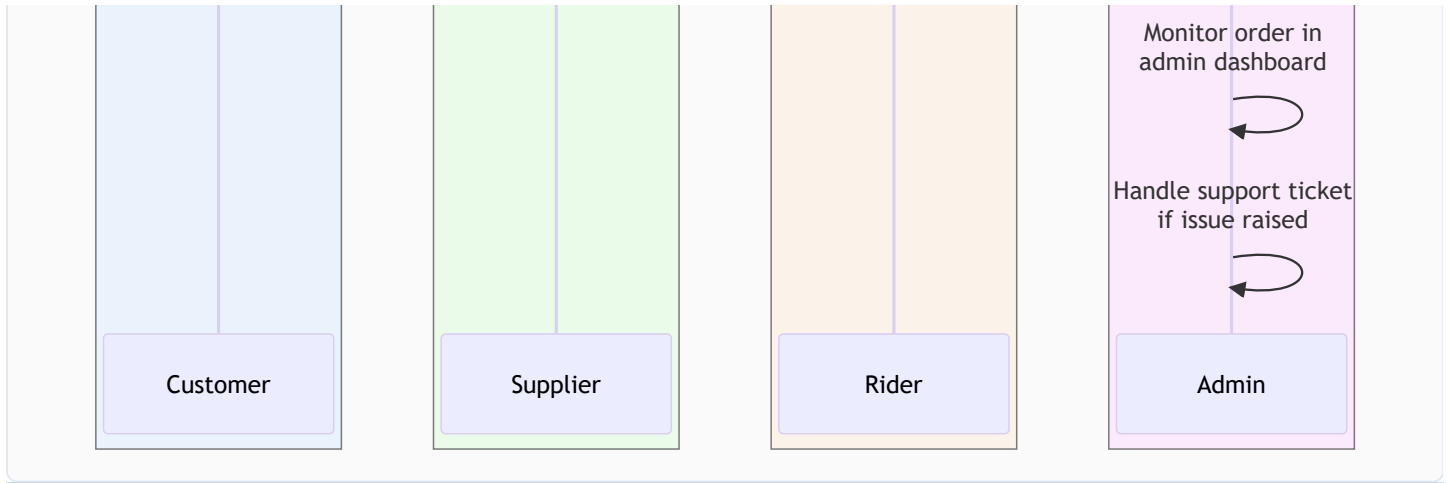
15. Multi-Actor Order Fulfillment Journey

This diagram shows how **one order** connects all actors from placement to delivery.









16. Key API Reference by Actor

Customer

Action	Method	Endpoint
Register / Login	POST	/api/auth/register , /api/auth/login
Profile	GET/PUT	/api/users/me
Addresses	CRUD	/api/addresses
Serviceability	POST	/api/serviceability/check
Products	GET	/api/products
Create order	POST	/api/orders
Razorpay pay	POST	/api/payments/razorpay/create-order , /verify-payment
Track order	GET	/api/orders/:id/tracking
Cancel order	PATCH	/api/orders/:id/cancel
Wallet	GET/POST	/api/wallet , /api/wallet/credit
AI insights	GET/POST	/api/ai/water-insight , /api/ai/ask
Support	CRUD	/api/customer-support/tickets

Supplier

Action	Method	Endpoint
Register	POST	/api/auth/register-supplier
Profile	GET/PATCH	/api/suppliers/me
Stores	CRUD	/api/stores
Products	CRUD	/api/products
Add rider	POST	/api/supplier/delivery-partners
Incoming orders	GET	/api/supplier/orders/incoming
Accept / Reject	PATCH	/api/supplier/orders/:id/accept , /reject
Assign rider	PATCH	/api/supplier/orders/:id/assign-rider
Financials	GET	/api/supplier/financials

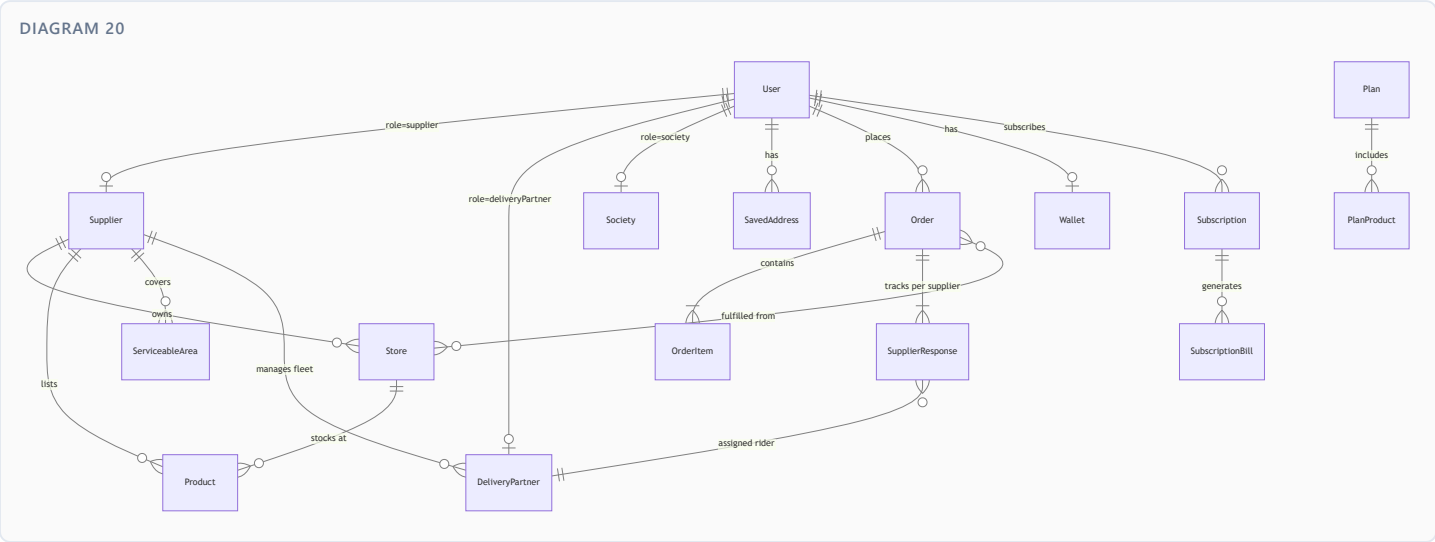
Delivery Partner (Rider)

Action	Method	Endpoint
Register	POST	/api/auth/register-delivery
Go online	PATCH	/api/delivery-partners/me/online
Share location	PATCH	/api/delivery-partners/me/location
Incoming orders	GET	/api/delivery-partners/orders/incoming
Pick up	PATCH	/api/delivery-partners/orders/:id/picked-up
Live GPS	PATCH	/api/delivery-partners/orders/:id/location
Deliver	PATCH	/api/delivery-partners/orders/:id/delivered

Admin

Action	Method	Endpoint
Login	POST	/api/admin/auth/login
Verify supplier	PATCH	/api/admin/suppliers/:id/verify
Verify rider	PATCH	/api/admin/delivery-partners/:id/verify
Approve store	PATCH	/api/admin/stores/:id/approve
Serviceable areas	CRUD	/api/admin/serviceable-areas
All orders	GET	/api/admin/orders
Financials	GET	/api/admin/financials
Admin users	CRUD	/api/admin/admins

17. Data Models (Core Entities)



Entity Summary

Model	File	Key Relationships
User	models/User.js	Central auth entity; roles: customer, supplier, deliveryPartner, society, admin, sub-admin
Supplier	models/Supplier.js	Linked to User; onboarding, commission, geo
DeliveryPartner	models/DeliveryPartner.js	Linked to User + optional Supplier fleet
Store	models/Store.js	Supplier location; approval workflow
Product	models/Product.js	Catalog item; supplier + store linkage
Order	models/Order.js	Full lifecycle; supplierResponses[], travelInfo[]
SavedAddress	models/SavedAddress.js	Customer delivery addresses
ServiceableArea	models/ServiceableArea.js	PIN + radius per supplier
Wallet	models/Wallet.js	Customer + platform balances
Subscription	models/Subscription.js	Recurring water plans

Related Documentation

Document	Path	Contents
Deployment Guide	docs/DEPLOYMENT-GUIDE.md	AWS EC2 setup, env vars, PM2
Admin Ubuntu Setup	admin/RUN-ON-UBUNTU.md	Admin portal deployment
Local Mobile Setup	mobile/LOCAL_SETUP.md	Expo + local backend
Customer Journey Catalog	docs/customer-journey-catalog.json	Screen-by-screen customer flow
AWS Production Config	config/aws-production.json	Production URLs

Diagram Rendering

All diagrams use Mermaid syntax. They render automatically in:

- [GitHub / GitLab markdown preview](#)
- [VS Code with a Mermaid extension](#)
- [Mermaid Live Editor](#)

For PDF export, use a Mermaid-capable markdown converter or paste diagrams into Mermaid Live Editor.